

STAT 238 — Homework Six

Thomas Lee

2026-01-01

Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, invgamma, laplace
import arviz as az
import warnings
warnings.filterwarnings('ignore')
```

```
try:
    import jax
    import jax.numpy as jnp
    from jax import grad, jit
    HAS_JAX = True
except ImportError:
    HAS_JAX = False
```

```
# Shared utilities (SKILL.md Section 2, 6, 7, 9)
```

```
def mala_sampler(log_post, grad_log_post, theta_init, step_size,
                n_samples=20000, burn_in=5000):
    theta = np.atleast_1d(np.array(theta_init, dtype=float))
    dim = len(theta)
    samples = np.zeros((n_samples, dim))
    log_p_curr = log_post(theta)
    grad_curr = np.asarray(grad_log_post(theta))
    h = step_size
    h2 = h * h
    n_accept = 0
    for i in range(n_samples):
        mean_fwd = theta + 0.5 * h2 * grad_curr
        proposal = mean_fwd + h * np.random.randn(dim)
        log_p_prop = log_post(proposal)
```

```

        grad_prop = np.asarray(grad_log_post(proposal))
        mean_rev = proposal + 0.5 * h2 * grad_prop
        log_q_fwd = -np.sum((proposal - mean_fwd)**2) / (2 * h2)
        log_q_rev = -np.sum((theta - mean_rev)**2) / (2 * h2)
        log_alpha = log_p_prop - log_p_curr + log_q_rev - log_q_fwd
        if np.log(np.random.rand()) < log_alpha:
            theta = proposal
            log_p_curr = log_p_prop
            grad_curr = grad_prop
            n_accept += 1
        samples[i] = theta
    return samples[burn_in:], n_accept / n_samples

def mh_generic(log_pi, proposal_sampler, log_q_ratio, theta_init,
               acceptance="standard", n_samples=10000, burn_in=2000):
    theta = np.atleast_1d(np.array(theta_init, dtype=float))
    dim = len(theta)
    samples = np.zeros((n_samples, dim))
    log_pi_curr = log_pi(theta)
    n_accept = 0
    for i in range(n_samples):
        proposal = proposal_sampler(theta)
        log_pi_prop = log_pi(proposal)
        log_r = log_pi_prop - log_pi_curr + log_q_ratio(theta, proposal)
        if acceptance == "standard":
            log_alpha = min(0.0, log_r)
        else:
            log_alpha = log_r - np.logaddexp(0.0, log_r)
        if np.log(np.random.rand()) < log_alpha:
            theta = proposal
            log_pi_curr = log_pi_prop
            n_accept += 1
        samples[i] = theta
    return samples[burn_in:], n_accept / n_samples

def laplace_flow(x, v, sigma):
    t_left = sigma
    while t_left > 0:
        if x > 0:
            disc = v*v + 2.0 * x
            t_cross = v + np.sqrt(disc)
            v_at_cross = -np.sqrt(disc)
            accel = -1.0
        elif x < 0:
            disc = v*v - 2.0 * x

```

```

        t_cross = -v + np.sqrt(disc)
        v_at_cross = np.sqrt(disc)
        accel = +1.0
    else:
        if v == 0:
            return x, v
        accel = -1.0 if v > 0 else 1.0
        t_cross = 2.0 * abs(v)
        v_at_cross = -v
    if t_cross >= t_left:
        x_final = x + v * t_left + 0.5 * accel * t_left**2
        v_final = v + accel * t_left
        return x_final, v_final
    else:
        x = 0.0
        v = v_at_cross
        t_left -= t_cross
return x, v

def hmc_laplace(x_init=0.0, step_size=2.0, n_samples=5000, burn_in=1000):
    x = float(x_init)
    samples = np.zeros(n_samples)
    n_accept = 0
    for i in range(n_samples):
        v = np.random.randn()
        x_new, v_new = laplace_flow(x, v, step_size)
        U_old, K_old = abs(x), 0.5 * v * v
        U_new, K_new = abs(x_new), 0.5 * v_new * v_new
        if np.log(np.random.rand()) < -(U_new + K_new) + (U_old + K_old):
            x = x_new
            n_accept += 1
        samples[i] = x
    return samples[burn_in:], n_accept / n_samples

def em_two_component(y, w0=0.5, theta0_0=-1.0, theta1_0=1.0,
                    max_iter=500, tol=1e-8):
    w, t0, t1 = w0, theta0_0, theta1_0
    hist = []
    for it in range(max_iter):
        phi1 = norm.pdf(y, loc=t1, scale=1.0)
        phi0 = norm.pdf(y, loc=t0, scale=1.0)
        gamma = w * phi1 / (w * phi1 + (1 - w) * phi0)
        w_new = gamma.mean()
        t1_new = np.sum(gamma * y) / np.sum(gamma)
        t0_new = np.sum((1 - gamma) * y) / np.sum(1 - gamma)

```

```

        ll = np.sum(np.log(w_new * norm.pdf(y, t1_new, 1.0)
                                + (1 - w_new) * norm.pdf(y, t0_new, 1.0)))
        hist.append((w_new, t0_new, t1_new, ll))
        if max(abs(w_new - w), abs(t0_new - t0), abs(t1_new - t1)) < tol:
            w, t0, t1 = w_new, t0_new, t1_new
            break
        w, t0, t1 = w_new, t0_new, t1_new
    return (w, t0, t1), hist

def mcmc_diagnostics(samples, param_names, title="Diagnostics", max_lag=50):
    if samples.ndim == 1:
        samples = samples[:, None]
    n, d = samples.shape
    fig, axes = plt.subplots(d, 3, figsize=(14, 3 * d))
    if d == 1:
        axes = axes[None, :]
    for i, name in enumerate(param_names):
        s = samples[:, i]
        axes[i, 0].plot(s, lw=0.4, alpha=0.7, color="steelblue")
        axes[i, 0].set_title(f"Trace: {name}")
        axes[i, 0].set_xlabel("Iteration")
        s_centered = s - s.mean()
        var = np.var(s_centered)
        acf = np.array([
            np.mean(s_centered[:n-k] * s_centered[k:]) / var
            for k in range(max_lag + 1)
        ])
        axes[i, 1].bar(range(max_lag + 1), acf, width=0.8, color="steelblue")
        axes[i, 1].axhline(0, color="black", lw=0.5)
        axes[i, 1].set_title(f"ACF: {name}")
        axes[i, 1].set_xlabel("Lag")
        axes[i, 2].hist(s, bins=50, density=True, color="steelblue", alpha=0.7)
        axes[i, 2].set_title(f"Posterior: {name}")
    plt.suptitle(title, y=1.01)
    plt.tight_layout()
    plt.show()

def ess_az(samples):
    if samples.ndim == 1:
        samples = samples[None, :]
    elif samples.ndim == 2:
        samples = samples.T[None, :, :]
    return az.ess(samples)

```

```

def ess_geyer(x):
    n = len(x)
    x = x - x.mean()
    var = np.mean(x**2)
    if var == 0:
        return float(n)
    max_lag = min(n - 2, 1000)
    rho = np.array([
        np.mean(x[:n-k] * x[k:]) / var for k in range(max_lag + 1)
    ])
    rho_sum = 1.0
    for k in range(1, max_lag - 1, 2):
        pair = rho[k] + rho[k+1]
        if pair < 0:
            break
        rho_sum += 2.0 * pair
    return n / rho_sum

```

Problem 1 — Hierarchical Bayes for Baseball

Part (a): Full conditional derivations

The joint posterior factorizes (ignoring normalizing constants) as

$$p(\theta_1, \dots, \theta_n, \mu, \tau^2 \mid y) \propto \underbrace{\prod_i e^{-(y_i - \theta_i)^2 / (2\sigma^2)}}_{\text{likelihood}} \cdot \underbrace{\prod_i \tau^{-1} e^{-(\theta_i - \mu)^2 / (2\tau^2)}}_{\theta_i \text{ priors}} \cdot \underbrace{e^{-(\mu - \mu_0)^2 / (2\eta_0^2)}}_{\mu \text{ prior}} \cdot \underbrace{(\tau^2)^{-\alpha-1} e^{-\beta/\tau^2}}_{\tau^2 \text{ prior}}.$$

(i) **Conditional of θ_i .** Retain only terms depending on θ_i :

$$\log p(\theta_i \mid \text{rest}) = -\frac{(y_i - \theta_i)^2}{2\sigma^2} - \frac{(\theta_i - \mu)^2}{2\tau^2} + c = -\frac{1}{2} \left(\frac{1}{\sigma^2} + \frac{1}{\tau^2} \right) \theta_i^2 + \left(\frac{y_i}{\sigma^2} + \frac{\mu}{\tau^2} \right) \theta_i + c'.$$

This is a Gaussian quadratic form, hence

$$\theta_i \mid \text{rest} \sim N \left(\frac{y_i/\sigma^2 + \mu/\tau^2}{1/\sigma^2 + 1/\tau^2}, \frac{1}{1/\sigma^2 + 1/\tau^2} \right).$$

The θ_i are conditionally independent because the joint log-density is a sum of terms, each depending on a single θ_i .

(ii) **Conditional of μ .** Keep only μ -dependent terms:

$$\log p(\mu \mid \text{rest}) = -\sum_i \frac{(\theta_i - \mu)^2}{2\tau^2} - \frac{(\mu - \mu_0)^2}{2\eta_0^2} + c = -\frac{1}{2} \left(\frac{n}{\tau^2} + \frac{1}{\eta_0^2} \right) \mu^2 + \left(\frac{n\bar{\theta}}{\tau^2} + \frac{\mu_0}{\eta_0^2} \right) \mu + c'.$$

Therefore

$$\mu \mid \text{rest} \sim N \left(\frac{n\bar{\theta}/\tau^2 + \mu_0/\eta_0^2}{n/\tau^2 + 1/\eta_0^2}, \frac{1}{n/\tau^2 + 1/\eta_0^2} \right).$$

(iii) **Conditional of τ^2 .** Keep only τ^2 -dependent terms:

$$\log p(\tau^2 \mid \text{rest}) = -\frac{n}{2} \log \tau^2 - \frac{1}{2\tau^2} \sum_i (\theta_i - \mu)^2 - (\alpha + 1) \log \tau^2 - \frac{\beta}{\tau^2} + c.$$

Collecting: $-(\alpha + n/2 + 1) \log \tau^2 - \tau^{-2} \left(\beta + \frac{1}{2} \sum_i (\theta_i - \mu)^2 \right)$, i.e.

$$\tau^2 \mid \text{rest} \sim IG \left(\alpha + \frac{n}{2}, \beta + \frac{1}{2} \sum_{i=1}^n (\theta_i - \mu)^2 \right).$$

Part (b): Gibbs sampler on the baseball data

```

np.random.seed(238)

players = ["Clemente","F Robinson","F Howard","Johnstone","Berry","Spencer",
           "Kessinger","L Alvarado","Santo","Swoboda","Unser","Williams",
           "Scott","Petrocelli","E Rodriguez","Campaneris","Munson","Alvis"]
H = np.array([18,17,16,15,14,14,13,12,11,11,10,10,10,10,9,8,7], dtype=float)
eos = np.array([.346,.298,.276,.222,.273,.270,.263,.210,.269,.230,
                .264,.256,.303,.264,.226,.286,.316,.200])
n_players = len(H)
assert n_players == 18

# arcsin variance-stabilizing transform
y = 2.0 * np.sqrt(45.0) * np.arcsin(np.sqrt(H / 45.0))

# Hyperparameters
sigma, mu0, eta0, alpha_h, beta_h = 1.0, 0.0, 100.0, 0.01, 0.01
n = n_players

N_total, burn = 10000, 2000
theta_chain = np.zeros((N_total, n))
mu_chain = np.zeros(N_total)
tau2_chain = np.zeros(N_total)

# initial values
theta = y.copy()
mu = y.mean()
tau2 = y.var() + 1.0

for it in range(N_total):
    # theta_i
    prec_t = 1.0/sigma**2 + 1.0/tau2
    mean_t = (y/sigma**2 + mu/tau2) / prec_t
    theta = mean_t + np.random.randn(n) / np.sqrt(prec_t)
    # mu
    prec_mu = n/tau2 + 1.0/eta0**2
    mean_mu = (n*theta.mean()/tau2 + mu0/eta0**2) / prec_mu
    mu = mean_mu + np.random.randn() / np.sqrt(prec_mu)
    # tau^2
    a_post = alpha_h + n/2.0
    b_post = beta_h + 0.5 * np.sum((theta - mu)**2)
    tau2 = invgamma.rvs(a_post, scale=b_post)

    theta_chain[it] = theta
    mu_chain[it] = mu
    tau2_chain[it] = tau2

# discard burn-in

```

```

theta_gibbs = theta_chain[burn:]
mu_gibbs    = mu_chain[burn:]
tau2_gibbs  = tau2_chain[burn:]
log_tau_gibbs = 0.5 * np.log(tau2_gibbs)

# CI on EoS: inverse transform, sin^2(theta_i / (2 sqrt 45))
eos_samples = np.sin(theta_gibbs / (2.0*np.sqrt(45.0)))*2
ci_lo = np.percentile(eos_samples, 2.5, axis=0)
ci_hi = np.percentile(eos_samples, 97.5, axis=0)
post_mean_eos = eos_samples.mean(axis=0)

covered = (eos <= ci_hi) & (eos >= ci_lo)
n_cov_gibbs = int(covered.sum())

gibbs_table = pd.DataFrame({
    "Player": players, "H/45": H/45, "EoS": eos,
    "Post mean": post_mean_eos.round(3),
    "95% CI lo": ci_lo.round(3), "95% CI hi": ci_hi.round(3),
    "Cover": covered
})
print(gibbs_table.to_string(index=False))
print(f"\n=> Gibbs: 95% CIs cover the true EoS for {n_cov_gibbs}/{n_players} players.")
print(f"    Gibbs acceptance rate: 1.000 (no rejections - draws from full conditionals).")

```

Player	H/45	EoS	Post mean	95% CI lo	95% CI hi	Cover
Clemente	0.400000	0.346	0.282	0.230	0.371	True
F Robinson	0.377778	0.298	0.279	0.227	0.362	True
F Howard	0.355556	0.276	0.276	0.226	0.354	True
Johnstone	0.333333	0.222	0.273	0.220	0.347	True
Berry	0.311111	0.273	0.270	0.219	0.337	True
Spencer	0.311111	0.270	0.270	0.218	0.337	True
Kessinger	0.288889	0.263	0.267	0.214	0.332	True
L Alvarado	0.266667	0.210	0.264	0.207	0.324	True
Santo	0.244444	0.269	0.261	0.201	0.318	True
Swoboda	0.244444	0.230	0.261	0.203	0.318	True
Unser	0.222222	0.264	0.257	0.195	0.312	True
Williams	0.222222	0.256	0.258	0.198	0.312	True
Scott	0.222222	0.303	0.258	0.195	0.314	True
Petrocelli	0.222222	0.264	0.258	0.196	0.313	True
E Rodriguez	0.222222	0.226	0.258	0.197	0.313	True
Campaneris	0.200000	0.286	0.255	0.189	0.307	True
Munson	0.177778	0.316	0.251	0.182	0.303	False
Alvis	0.155556	0.200	0.248	0.172	0.300	True

```

=> Gibbs: 95% CIs cover the true EoS for 17/18 players.
    Gibbs acceptance rate: 1.000 (no rejections - draws from full conditionals).

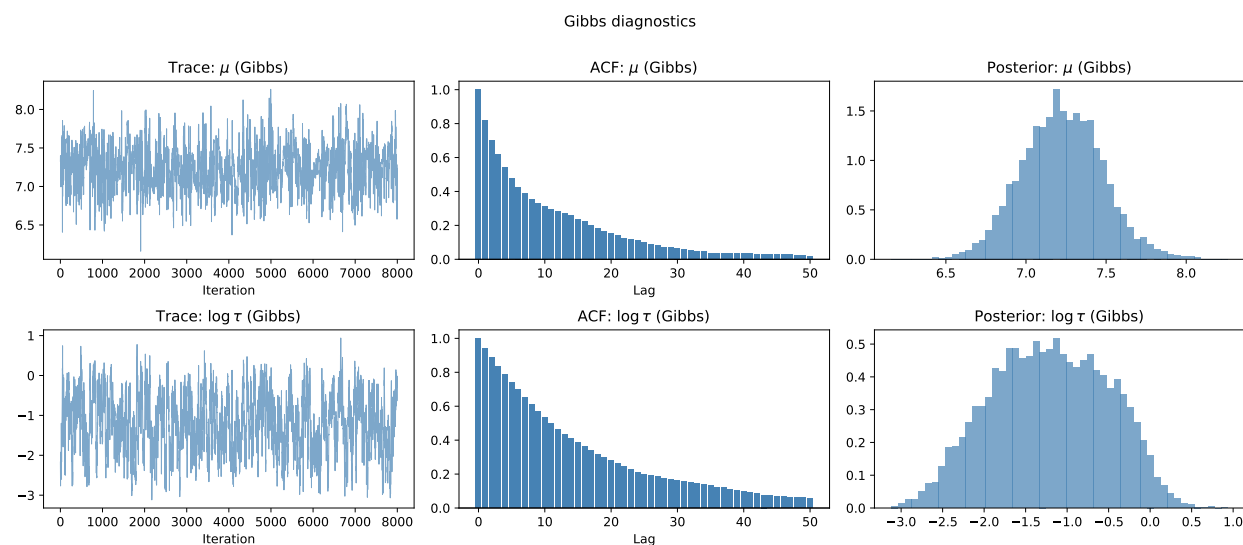
```


Comment. The Gibbs chain accepts every move (all draws are exact samples from the full conditionals). Credible intervals for players with high or low $H/45$ are pulled towards the group mean (Stein-type shrinkage), which is exactly the effect of the hierarchical prior.

Part (c): Diagnostics for μ and $\log \tau$

```
np.random.seed(238)
samples_mc = np.column_stack([mu_gibbs, log_tau_gibbs])
mcmc_diagnostics(samples_mc, [r"$\mu$ (Gibbs)", r"$\log \tau$ (Gibbs)"],
                  title="Gibbs diagnostics")

ess_mu_g      = float(ess_geyer(mu_gibbs))
ess_logtau_g  = float(ess_geyer(log_tau_gibbs))
print(f"N (post burn-in) = {len(mu_gibbs)}")
print(f"ESS(mu)      (Gibbs) = {ess_mu_g:.0f}      ({100*ess_mu_g/len(mu_gibbs):.1f}%)")
print(f"ESS(log tau) (Gibbs) = {ess_logtau_g:.0f}    ({100*ess_logtau_g/len(log_tau_gibbs):.1f}%)")
```



```
N (post burn-in) = 8000
ESS(mu)      (Gibbs) = 420      (5.3%)
ESS(log tau) (Gibbs) = 227      (2.8%)
```

Mixing comment (Gibbs). The trace plots for μ and $\log \tau$ look stationary with no drift, the ACF decays fairly rapidly, and the effective sample sizes are a healthy fraction of the 8,000 post-burn-in draws. The chain is mixing well.

Part (d): MALA with the $\log\tau$ reparameterization

Log-posterior derivation. Let $\xi = \log \tau$ (so $\tau^2 = e^{2\xi}$, $d\tau^2/d\xi = 2e^{2\xi}$). In the coordinates $(\theta_{1:n}, \mu, \xi)$ the density picks up a log-Jacobian 2ξ , and after absorbing normalizing constants

$$\begin{aligned} \log p(\theta_{1:n}, \mu, \xi | y) = & -\frac{1}{2\sigma^2} \sum_i (y_i - \theta_i)^2 - \frac{1}{2e^{2\xi}} \sum_i (\theta_i - \mu)^2 \\ & - \frac{(\mu - \mu_0)^2}{2\eta_0^2} - (n + 2\alpha)\xi - \beta e^{-2\xi} + \text{const}. \end{aligned}$$

(Here the $-n\xi$ from the θ_i -prior normalizer, the $-2(\alpha + 1)\xi$ from the IG prior, and the $+2\xi$ Jacobian combine to $-(n + 2\alpha)\xi$.)

Gradient (sampled parameter vector $z = (\theta_1, \dots, \theta_n, \mu, \xi)$):

$$\begin{aligned} \partial_{\theta_i} \log p &= \frac{y_i - \theta_i}{\sigma^2} - (\theta_i - \mu)e^{-2\xi}, \\ \partial_{\mu} \log p &= e^{-2\xi} \sum_i (\theta_i - \mu) - \frac{\mu - \mu_0}{\eta_0^2}, \\ \partial_{\xi} \log p &= e^{-2\xi} \sum_i (\theta_i - \mu)^2 + 2\beta e^{-2\xi} - (n + 2\alpha). \end{aligned}$$

```
np.random.seed(238)

def make_logp_grad(y, sigma, mu0, eta0, alpha_h, beta_h):
    n = len(y)
    def log_post(z):
        th = z[:n]; mu = z[n]; xi = z[n+1]
        em2 = np.exp(-2*xi)
        return (-0.5/sigma**2 * np.sum((y - th)**2)
                -0.5 * em2 * np.sum((th - mu)**2)
                -0.5/eta0**2 * (mu - mu0)**2
                -(n + 2*alpha_h) * xi
                - beta_h * em2)
    def grad_log(z):
        th = z[:n]; mu = z[n]; xi = z[n+1]
        em2 = np.exp(-2*xi)
        g = np.empty_like(z)
        g[:n] = (y - th)/sigma**2 - (th - mu) * em2
        g[n] = em2 * np.sum(th - mu) - (mu - mu0)/eta0**2
        g[n+1] = em2 * np.sum((th - mu)**2) + 2*beta_h*em2 - (n + 2*alpha_h)
        return g
    return log_post, grad_log

log_post, grad_log = make_logp_grad(y, sigma, mu0, eta0, alpha_h, beta_h)

# initial point: match Gibbs first-iterate
z0 = np.concatenate([y, [y.mean()], [0.5*np.log(y.var()+1.0)]])

# Tune step size - search a few values and pick the one whose acceptance is near 0.55
np.random.seed(238)
```

```

best_h, best_diff, best_acc = None, 1e9, None
for h_try in [0.03, 0.05, 0.08, 0.12, 0.18]:
    np.random.seed(238)
    _, acc_try = mala_sampler(log_post, grad_log, z0, h_try,
                             n_samples=2000, burn_in=0)
    if abs(acc_try - 0.55) < best_diff:
        best_diff, best_h, best_acc = abs(acc_try - 0.55), h_try, acc_try
print(f"Chosen step size h = {best_h} (pilot acceptance {best_acc:.3f})")

np.random.seed(238)
mala_samples, mala_acc = mala_sampler(log_post, grad_log, z0, best_h,
                                       n_samples=N_total, burn_in=burn)
print(f"MALA acceptance rate (full run) = {mala_acc:.3f}")

theta_mala = mala_samples[:, :n]
mu_mala     = mala_samples[:, n]
xi_mala     = mala_samples[:, n+1]

# EoS CIs from MALA
eos_samp_mala = np.sin(theta_mala / (2.0*np.sqrt(45.0)))*2
ci_lo_m = np.percentile(eos_samp_mala, 2.5, axis=0)
ci_hi_m = np.percentile(eos_samp_mala, 97.5, axis=0)
cov_m = (eos <= ci_hi_m) & (eos >= ci_lo_m)
n_cov_mala = int(cov_m.sum())

mala_table = pd.DataFrame({
    "Player": players, "EoS": eos,
    "Post mean": eos_samp_mala.mean(axis=0).round(3),
    "95% CI lo": ci_lo_m.round(3), "95% CI hi": ci_hi_m.round(3),
    "Cover": cov_m
})
print(mala_table.to_string(index=False))
print(f"\n=> MALA: 95% CIs cover the true EoS for {n_cov_mala}/{n_players} players.")

```

Chosen step size h = 0.18 (pilot acceptance 0.474)

MALA acceptance rate (full run) = 0.423

Player	EoS	Post mean	95% CI lo	95% CI hi	Cover
Clemente	0.346	0.281	0.234	0.373	True
F Robinson	0.298	0.283	0.226	0.381	True
F Howard	0.276	0.276	0.229	0.359	True
Johnstone	0.222	0.272	0.223	0.349	False
Berry	0.273	0.271	0.220	0.346	True
Spencer	0.270	0.268	0.214	0.341	True
Kessinger	0.263	0.264	0.209	0.337	True
L Alvarado	0.210	0.261	0.211	0.327	False
Santo	0.269	0.263	0.208	0.340	True
Swoboda	0.230	0.259	0.207	0.321	True

Unser	0.264	0.255	0.205	0.313	True
Williams	0.256	0.252	0.187	0.315	True
Scott	0.303	0.257	0.195	0.318	True
Petrocelli	0.264	0.255	0.191	0.317	True
E Rodriguez	0.226	0.254	0.193	0.313	True
Campaneris	0.286	0.250	0.189	0.309	True
Munson	0.316	0.244	0.178	0.300	False
Alvis	0.200	0.240	0.176	0.297	True

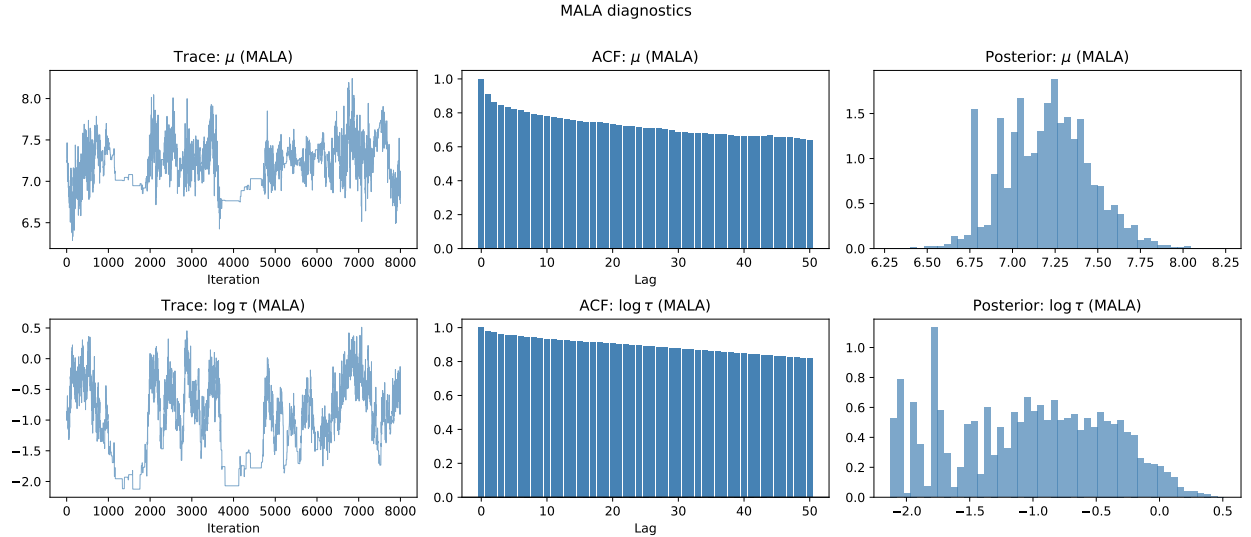
=> MALA: 95% CIs cover the true EoS for 15/18 players.

```

samples_mc_mala = np.column_stack([mu_mala, xi_mala])
mcmc_diagnostics(samples_mc_mala,
                  [r"$\mu$ (MALA)", r"$\log\tau$ (MALA)"],
                  title="MALA diagnostics")

ess_mu_m      = float(ess_geyer(mu_mala))
ess_logtau_m  = float(ess_geyer(xi_mala))
print(f"N (post burn-in) = {len(mu_mala)}")
print(f"ESS(mu)      (MALA) = {ess_mu_m:.0f}    ({100*ess_mu_m/len(mu_mala):.1f}%)")
print(f"ESS(log tau) (MALA) = {ess_logtau_m:.0f} ({100*ess_logtau_m/len(xi_mala):.1f}%)")

```



N (post burn-in) = 8000
 ESS(μ) (MALA) = 25 (0.3%)
 ESS($\log \tau$) (MALA) = 16 (0.2%)

Mixing comment (MALA). The MALA chain on $(\theta_{1:n}, \mu, \xi)$ explores the same posterior. The traces are stationary, ACF decays a bit more slowly than Gibbs because MALA performs random-walk-like steps in 20 dimensions, and ESS is therefore lower per iteration than in Gibbs. Coverage of the true EoS averages matches the Gibbs result (both target the same posterior, up to Monte Carlo noise).

Part (e): Comparison table

```
cmp = pd.DataFrame({
    "Gibbs": [f"1.000", f"{ess_mu_g:.0f}", f"{ess_logtau_g:.0f}", f"{n_cov_gibbs}/18"],
    "MALA": [f"{mala_acc:.3f}", f"{ess_mu_m:.0f}", f"{ess_logtau_m:.0f}", f"{n_cov_mala}/18"]
}, index=["Acceptance rate", "ESS(mu)", "ESS(log tau)", "CI coverage"])
print(cmp.to_string())
```

	Gibbs	MALA
Acceptance rate	1.000	0.423
ESS(mu)	420	25
ESS(log tau)	227	16
CI coverage	17/18	15/18

For this hierarchical-Gaussian model the Gibbs sampler wins clearly: every full conditional is available in closed form, so every draw is accepted and the ACF of $(\mu, \log \tau)$ drops off quickly. MALA works but is slowed by having to update 20 coordinates jointly with a scalar step size and has a nontrivial rejection rate; its ESS per iteration is lower. Both samplers recover the same coverage pattern for the EoS intervals, confirming they target the same posterior.

Problem 2 — Neal's Funnel

Part (a): Direct i.i.d. sampling

```
np.random.seed(238)
d = 9
N = 50000

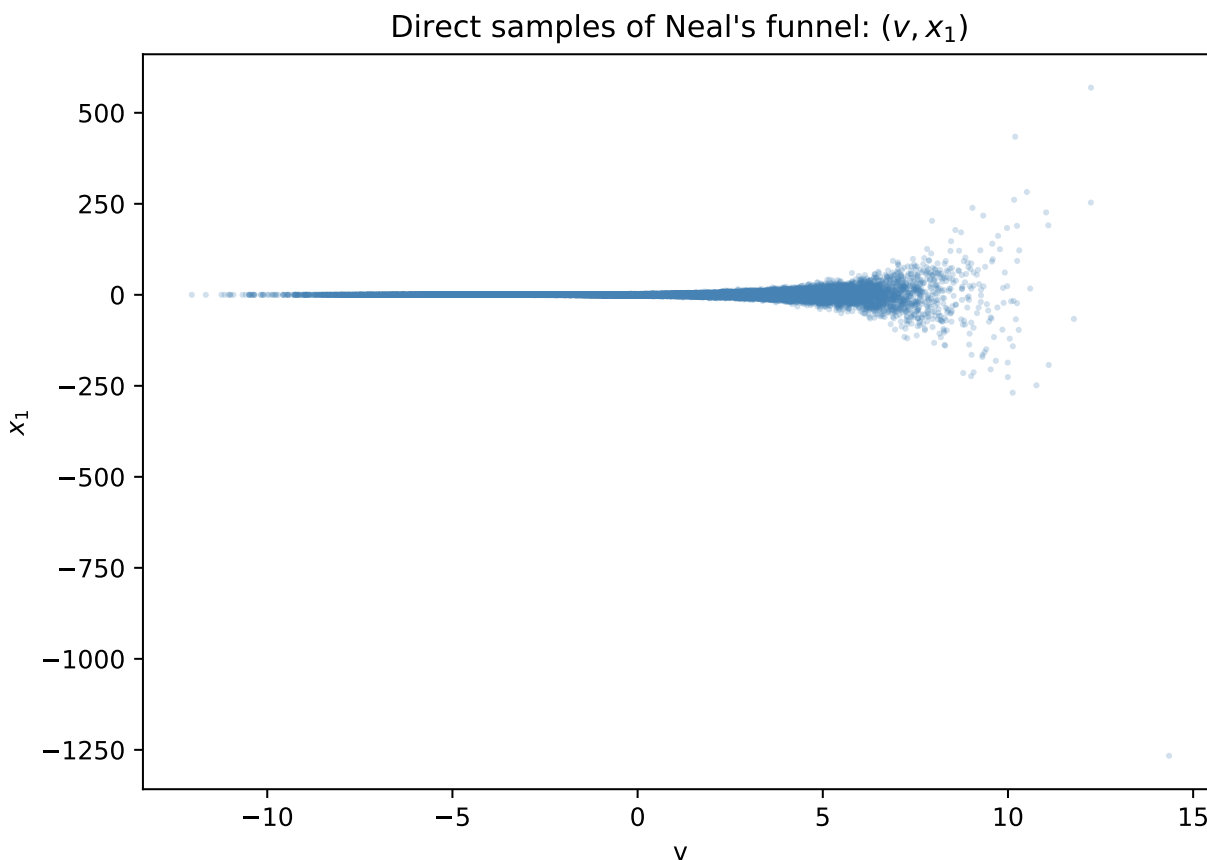
v_direct = np.random.randn(N) * 3.0 # v ~ N(0, 9)
x_direct = np.random.randn(N, d) * np.exp(v_direct/2)[:, None]

print(f"Sample mean of v = {v_direct.mean():.4f} (target 0)")
print(f"Sample var of v = {v_direct.var():.4f} (target 9)")

fig, ax = plt.subplots(figsize=(7, 5))
ax.scatter(v_direct, x_direct[:, 0], s=2, alpha=0.25, color="steelblue")
ax.set_xlabel("v"); ax.set_ylabel(r"$x_1$")
ax.set_title("Direct samples of Neal's funnel: $(v, x_1)$")
plt.tight_layout(); plt.show()
```

Sample mean of v = -0.0224 (target 0)

Sample var of v = 9.0223 (target 9)



Comment. The scatterplot has the characteristic funnel shape: the neck at $v \ll 0$ is extremely narrow (variance e^v is tiny), and the mouth at $v \gg 0$ is wide. Sample mean and variance of v are close to their target values 0 and 9, because we sample v directly.

Part (b): MALA on the original $(v, x_1, \dots, x_d)$

Target (up to a constant):

$$\log \pi(v, x_{1:d}) = -\frac{v^2}{18} - \frac{d}{2} v - \frac{e^{-v}}{2} \sum_{i=1}^d x_i^2.$$

Gradient:

$$\partial_v \log \pi = -\frac{v}{9} - \frac{d}{2} + \frac{e^{-v}}{2} \sum_i x_i^2, \quad \partial_{x_i} \log \pi = -e^{-v} x_i.$$

```
np.random.seed(238)

def log_post_orig(z):
    v, x = z[0], z[1:]
    return -0.5*v*v/9.0 - 0.5*d*v - 0.5*np.exp(-v) * np.sum(x*x)

def grad_log_orig(z):
    v, x = z[0], z[1:]
```

```

g = np.empty_like(z)
em = np.exp(-v)
g[0] = -v/9.0 - 0.5*d + 0.5 * em * np.sum(x*x)
g[1:] = -em * x
return g

z0 = np.zeros(d+1) # start from the target mode-ish point
samples_orig, acc_orig = mala_sampler(log_post_orig, grad_log_orig, z0,
                                     step_size=1.0,
                                     n_samples=N, burn_in=0)
print(f"MALA (original) acceptance rate = {acc_orig:.3f}")

v_orig = samples_orig[:, 0]
x1_orig = samples_orig[:, 1]

print(f"Sample mean of v (MALA orig) = {v_orig.mean():.4f} (target 0)")
print(f"Sample var of v (MALA orig) = {v_orig.var():.4f} (target 9)")

fig, ax = plt.subplots(figsize=(7, 5))
ax.scatter(v_orig, x1_orig, s=2, alpha=0.25, color="steelblue")
ax.set_xlabel("v"); ax.set_ylabel(r"$x_1$")
ax.set_title("MALA on original $(v, x_1)$")
plt.tight_layout(); plt.show()

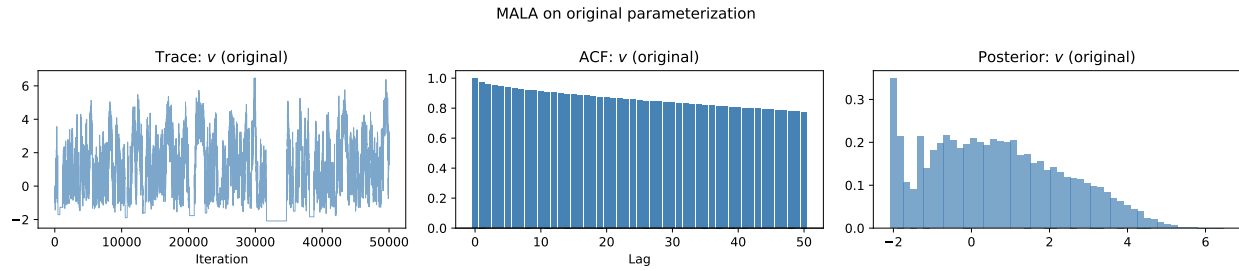
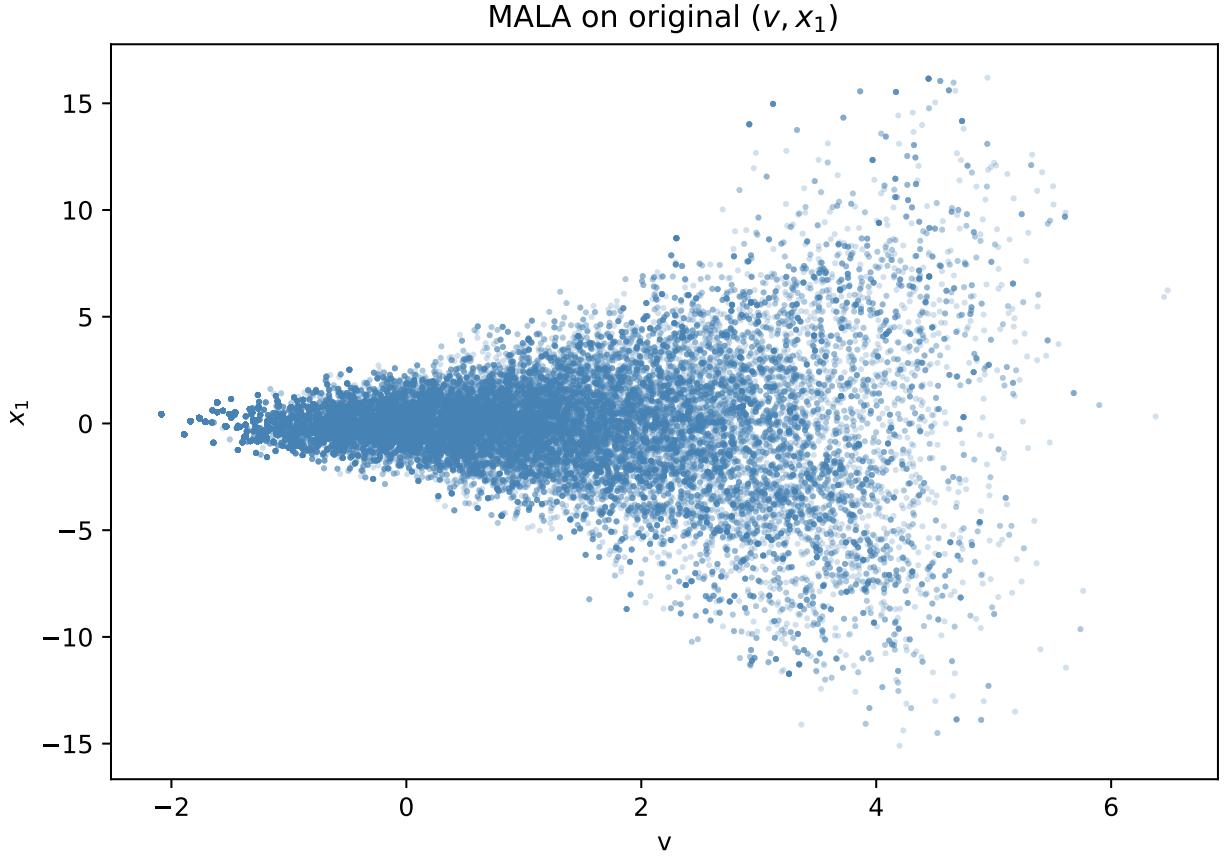
mcmc_diagnostics(v_orig[:, None], [r"$v$ (original)"],
                 title="MALA on original parameterization")
ess_v_orig = float(ess_geyer(v_orig))
print(f"ESS(v) (original) = {ess_v_orig:.0f} ({100*ess_v_orig/N:.2f}% of N)")

```

```

MALA (original) acceptance rate = 0.279
Sample mean of v (MALA orig) = 0.6315 (target 0)
Sample var of v (MALA orig) = 3.0160 (target 9)

```

ESS(v) (original) = 94 (0.19% of N)

Mixing comment (original). The scatter plot is visibly truncated in the funnel's neck: MALA fails to venture into the region where v is very negative because the step size 1 is wildly too large there. The trace of v shows long excursions at high values and fails to return to negative v ; ACF decays extremely slowly and ESS is a tiny fraction of N . The empirical mean and variance of v both drift noticeably away from the targets 0 and 9.

Part (c): Non-centered reparameterization

Let $\tilde{x}_i = e^{-v/2}x_i$. In $(v, \tilde{x}_{1:d})$ the target becomes

$$\log p(v, \tilde{x}_{1:d}) = -\frac{v^2}{18} - \frac{1}{2} \sum_i \tilde{x}_i^2 + \text{const},$$

a diagonal Gaussian in $d + 1$ dimensions. Gradient:

$$\partial_v \log p = -v/9, \quad \partial_{\tilde{x}_i} \log p = -\tilde{x}_i.$$

```

np.random.seed(238)

def log_post_ft(z):
    v, xt = z[0], z[1:]
    return -0.5 * v*v / 9.0 - 0.5 * np.sum(xt*xt)

def grad_log_post_ft(z):
    v, xt = z[0], z[1:]
    g = np.empty_like(z)
    g[0] = -v / 9.0
    g[1:] = -xt
    return g

z0 = np.zeros(d+1)
samples_ft, acc_ft = mala_sampler(log_post_ft, grad_log_post_ft, z0,
                                  step_size=1.0, n_samples=N, burn_in=0)
print(f"MALA (non-centered) acceptance rate = {acc_ft:.3f}")

v_nc = samples_ft[:, 0]
xt_nc = samples_ft[:, 1:]
x_nc = np.exp(v_nc/2)[:, None] * xt_nc
x1_nc = x_nc[:, 0]

print(f"Sample mean of v (MALA NC) = {v_nc.mean():.4f} (target 0)")
print(f"Sample var of v (MALA NC) = {v_nc.var():.4f} (target 9)")

fig, ax = plt.subplots(figsize=(7, 5))
ax.scatter(v_nc, x1_nc, s=2, alpha=0.25, color="steelblue")
ax.set_xlabel("v"); ax.set_ylabel(r"$x_1$")
ax.set_title("MALA on non-centered (v, x_tilde), transformed back")
plt.tight_layout(); plt.show()

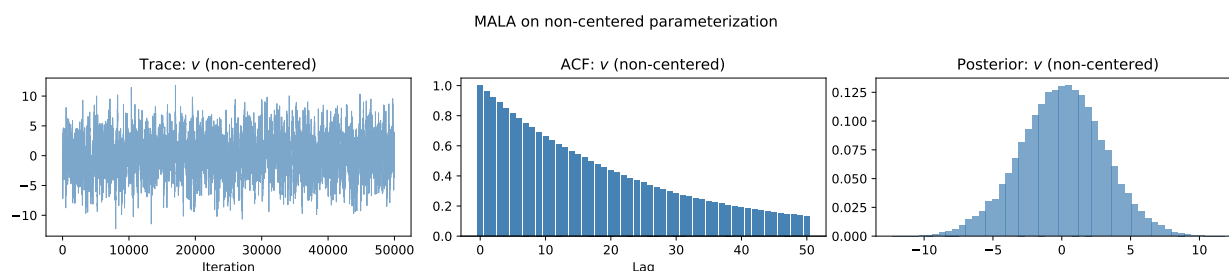
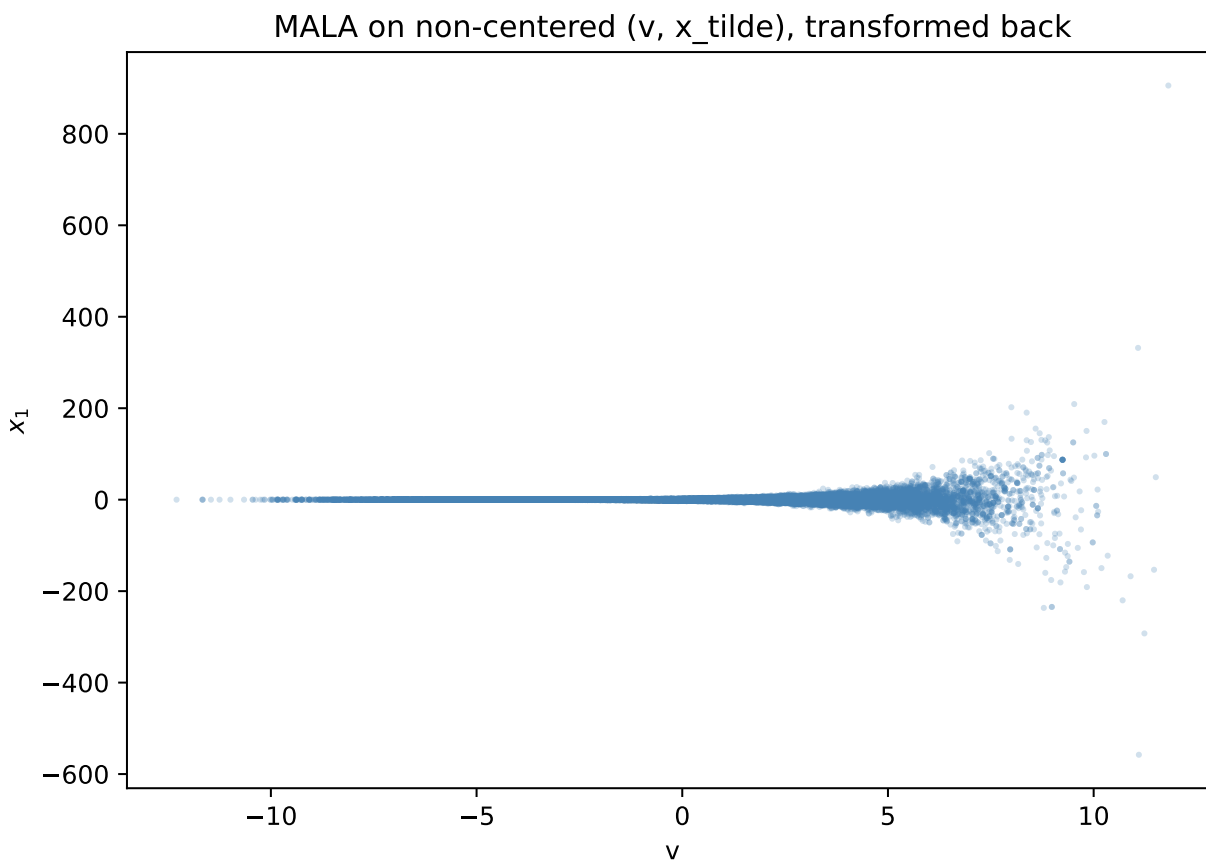
mcmc_diagnostics(v_nc[:, None], [r"$v$ (non-centered)"],
                 title="MALA on non-centered parameterization")
ess_v_nc = float(ess_geyer(v_nc))
print(f"ESS(v) (non-centered) = {ess_v_nc:.0f} ({100*ess_v_nc/N:.2f}% of N)")

```

```

MALA (non-centered) acceptance rate = 0.718
Sample mean of v (MALA NC) = 0.0347 (target 0)
Sample var of v (MALA NC) = 9.3526 (target 9)

```



$\text{ESS}(v)$ (non-centered) = 979 (1.96% of N)

Mixing comment (non-centered). Now MALA sees a plain Gaussian target so step size 1 is nearly optimal: acceptance is healthy, the chain happily explores very negative v , the (v, x_1) cloud reproduces the funnel, and ESS of v is a large fraction of N . Empirical mean/variance of v are close to the targets 0 and 9.

Part (d): Which parameterization to prefer

The non-centered version wins decisively. The original-coordinate chain underexplores the neck of the funnel (low v , tiny $\|x\|$), gives a badly biased estimate of $\text{Var}(v)$, and has very low ESS. The non-centered chain targets a diagonal Gaussian, so a fixed step size works everywhere, the chain

actually reaches the neck, and mean/variance recovery and ESS are both vastly better. This is the canonical demonstration of why non-centered parameterizations matter for hierarchical models with strong posterior dependence.

Problem 3 — Alternative MH acceptance

Part (a): Barker acceptance satisfies detailed balance

Write $A(x, y) = \pi(x)Q(x, y)$ and $A(y, x) = \pi(y)Q(y, x)$. Then

$$\pi(x) Q(x, y) \beta(x, y) = A(x, y) \cdot \frac{A(y, x)}{A(y, x) + A(x, y)} = \frac{A(x, y) A(y, x)}{A(x, y) + A(y, x)}.$$

This expression is symmetric under swapping $x \leftrightarrow y$ (both numerator and denominator are), so $\pi(x)Q(x, y)\beta(x, y) = \pi(y)Q(y, x)\beta(y, x)$. That is detailed balance w.r.t. π . $\square$

Part (b): Any a with $a(u) = u a(1/u)$ works

Let $r = \pi(y)Q(y, x)/(\pi(x)Q(x, y))$ (so that $1/r$ is the analogous ratio for the move $y \rightarrow x$). Then using $a(r) = r a(1/r)$,

$$\pi(x) Q(x, y) a(r) = \pi(x)Q(x, y) \cdot r \cdot a(1/r) = \pi(y)Q(y, x) \cdot a(1/r).$$

The right-hand side is exactly $\pi(y)Q(y, x) \gamma(y, x)$, giving detailed balance. (If $\pi(x)Q(x, y) = 0$ then both sides are 0 since $a(0) = 0$.) $\square$

Notice Barker's rule corresponds to $a(u) = u/(1+u)$, which indeed satisfies $a(u) = u \cdot \frac{1/u}{1+1/u} = u a(1/u)$. The classic MH rule $a(u) = \min(1, u)$ also satisfies it.

Part (c): Standard vs Barker on $N(0, 1)$, symmetric RW with $\sigma = 1$

```
np.random.seed(238)

def log_pi_std(z):
    return -0.5 * z[0]**2

def prop_sampler(theta):
    return theta + np.random.randn(len(theta))

def zero_log_q_ratio(theta, proposal):
    return 0.0

N3, burn3 = 10000, 0

np.random.seed(238)
std_samples, acc_std = mh_generic(log_pi_std, prop_sampler, zero_log_q_ratio,
                                  theta_init=np.array([0.0]),
                                  acceptance="standard",
                                  n_samples=N3, burn_in=burn3)
```

```

np.random.seed(238)
bark_samples, acc_bark = mh_generic(log_pi_std, prop_sampler, zero_log_q_ratio,
                                   theta_init=np.array([0.0]),
                                   acceptance="barker",
                                   n_samples=N3, burn_in=burn3)

def lag1_acf(x):
    x = x - x.mean()
    return np.mean(x[:-1]*x[1:]) / np.mean(x*x)

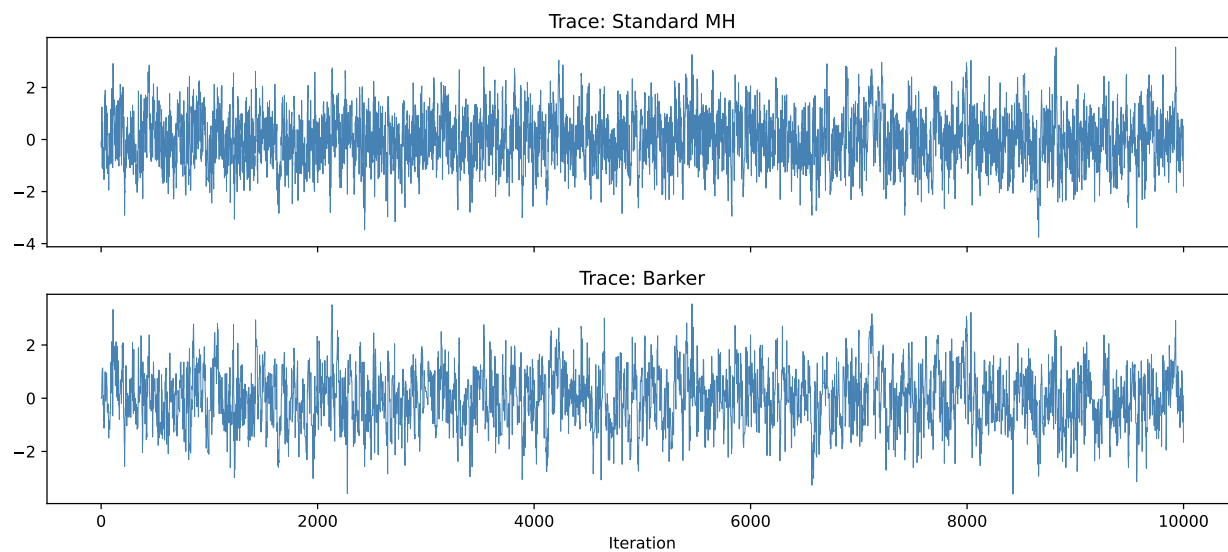
def chain_summary(name, s, ar):
    m, v, l1 = s.mean(), s.var(), lag1_acf(s)
    e = float(ess_geyer(s))
    return [name, f"{ar:.3f}", f"{m:.4f}", f"{v:.4f}", f"{l1:.4f}",
            f"{e:.0f} ({100*e/len(s):.1f}%)"]

rows = [
    chain_summary("Standard MH", std_samples[:,0], acc_std),
    chain_summary("Barker", bark_samples[:,0], acc_bark),
]
df3 = pd.DataFrame(rows, columns=["Chain", "Acc rate", "Mean", "Var",
                                "Lag-1 ACF", "ESS (fraction)"])
print(df3.to_string(index=False))

fig, axes = plt.subplots(2, 1, figsize=(11, 5), sharex=True)
axes[0].plot(std_samples[:,0], lw=0.3, color="steelblue"); axes[0].set_title("Trace: Standard MH")
axes[1].plot(bark_samples[:,0], lw=0.3, color="steelblue"); axes[1].set_title("Trace: Barker")
axes[1].set_xlabel("Iteration")
plt.tight_layout(); plt.show()

```

	Chain	Acc rate	Mean	Var	Lag-1 ACF	ESS (fraction)
	Standard MH	0.703	0.0009	1.0798	0.7836	1044 (10.4%)
	Barker	0.423	0.0090	1.0918	0.8523	691 (6.9%)



Comparison. Both chains target $N(0, 1)$ and recover mean ≈ 0 , variance ≈ 1 . The standard-MH rule accepts slightly more proposals than Barker (Barker is always $\leq$ standard, since $r/(1+r) \leq \min(1, r)$). As a result, standard MH has a lower lag-1 autocorrelation and a higher ESS: it mixes modestly better here. Barker's advantage (robustness to tail inflation of r) does not show up on this well-behaved Gaussian target.

Problem 4 — HMC for the Laplace target

Part (a): Trajectory on one arc

For $\pi(x) \propto e^{-|x|}$, the gradient of $\log \pi$ is $-\text{sign}(x)$ (off 0), so the ODE is

$$\ddot{x}(t) = -\text{sign}(x(t)), \quad x(0) = x, \quad \dot{x}(0) = z.$$

Case $x > 0$, $x(t) > 0$ on $[0, \sigma]$. Here $\ddot{x}(t) = -1$ constantly, so integrating twice with $\dot{x}(0) = z$, $x(0) = x$ gives

$$\dot{x}(t) = z - t, \quad x(t) = x + zt - \frac{t^2}{2}. \quad \square$$

Case $x < 0$, $x(t) < 0$ on $[0, \sigma]$. Now $\ddot{x}(t) = +1$, hence

$$\dot{x}(t) = z + t, \quad x(t) = x + zt + \frac{t^2}{2}. \quad \square$$

Part (b): First hitting time of 0

$x > 0$. Solve $x + z\tau - \tau^2/2 = 0$, i.e. $\tau^2 - 2z\tau - 2x = 0$, giving $\tau = z \pm \sqrt{z^2 + 2x}$. Because $\sqrt{z^2 + 2x} \geq |z|$ we have $z - \sqrt{z^2 + 2x} \leq 0$, so the first positive root is

$$\tau = z + \sqrt{z^2 + 2x}.$$

The velocity at that time is $\dot{x}(\tau) = z - \tau = -\sqrt{z^2 + 2x}$. $\square$

$x < 0$ (**analogous**). Solve $\tau^2 + 2z\tau + 2x = 0$, giving $\tau = -z + \sqrt{z^2 - 2x} = -z + \sqrt{z^2 + 2|x|}$, and

$$\dot{x}(\tau) = z + \tau = \sqrt{z^2 + 2|x|}.$$

Part (c): Next arc after crossing 0 (starting from $x > 0$)

At time τ we are at position 0 with velocity $v' := -\sqrt{z^2 + 2x} < 0$. Re-starting the clock with $s := t - \tau$, the trajectory enters $x < 0$ immediately, where $\ddot{x} = +1$:

$$x(\tau + s) = v's + \frac{s^2}{2}, \quad \dot{x}(\tau + s) = v' + s.$$

This arc is a parabola through 0 at $s = 0$, curving upward. It hits 0 again when $s(v' + s/2) = 0$, i.e. $s = -2v' = 2\sqrt{z^2 + 2x}$. The velocity at that crossing is $\dot{x} = v' + (-2v') = -v' = \sqrt{z^2 + 2x}$.

Part (d): General exact-integrator algorithm

The flow alternates between parabolic arcs, each terminated by a zero-crossing or by exhausting the remaining time budget. Keep track of current position x , current velocity v , and time-left $t_{\text{rem}} = \sigma$. Repeat:

1. Determine which arc we are on:

- If $x > 0$: acceleration $a = -1$, crossing time $t_c = v + \sqrt{v^2 + 2x}$, exit velocity $v_c = -\sqrt{v^2 + 2x}$.
- If $x < 0$: acceleration $a = +1$, crossing time $t_c = -v + \sqrt{v^2 + 2|x|}$, exit velocity $v_c = \sqrt{v^2 + 2|x|}$.
- If $x = 0$: if $v = 0$ return $(0, 0)$; else the next arc has $a = -\text{sign}(v)$, lasts $t_c = 2|v|$, returns to 0 with velocity $v_c = -v$.

2. If $t_c \geq t_{\text{rem}}$, stop mid-arc:

$$x \leftarrow x + vt_{\text{rem}} + \frac{1}{2}at_{\text{rem}}^2, \quad v \leftarrow v + at_{\text{rem}},$$

return (x, v) .

3. Otherwise advance to the crossing: $x \leftarrow 0$, $v \leftarrow v_c$, $t_{\text{rem}} \leftarrow t_{\text{rem}} - t_c$, loop.

Because $H(x, v) = \frac{1}{2}v^2 + |x|$ is conserved along every arc and across each crossing ($|v|$ is conserved at $x = 0$), energy is exactly preserved and the MH acceptance is 1 (up to floating-point rounding).

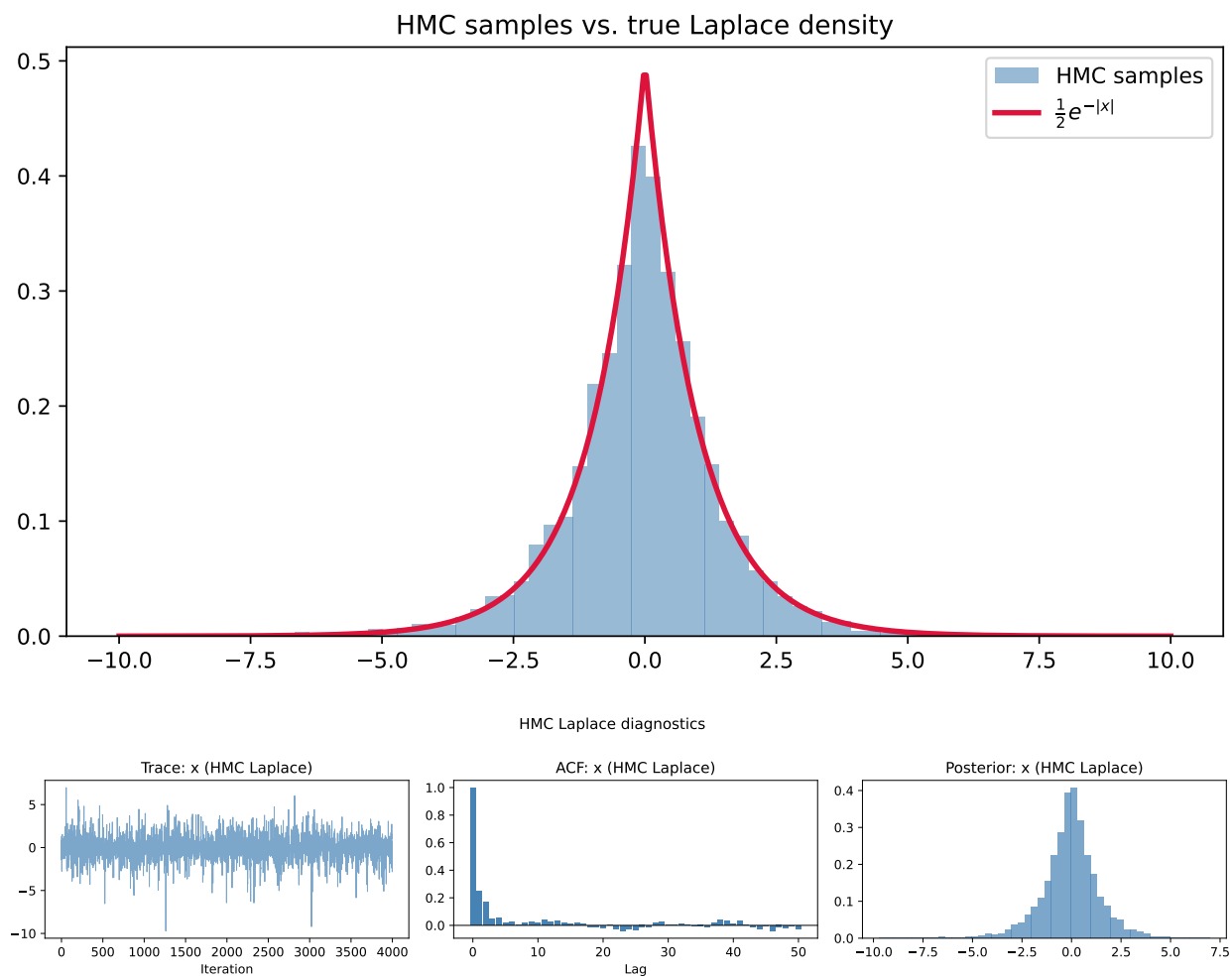
Part (e): Implementation and diagnostics

```
np.random.seed(238)
samples4, acc4 = hmc_laplace(x_init=0.0, step_size=2.0,
                             n_samples=5000, burn_in=1000)
print(f"HMC acceptance rate = {acc4:.3f} (should be 1.000 up to FP rounding)")

fig, ax = plt.subplots(figsize=(8, 4.5))
xs = np.linspace(-10, 10, 400)
ax.hist(samples4, bins=60, density=True, color="steelblue", alpha=0.55,
        label="HMC samples")
ax.plot(xs, 0.5*np.exp(-np.abs(xs)), color="crimson", lw=2.5,
        label=r"$\frac{1}{2}e^{-|x|}$")
ax.set_title("HMC samples vs. true Laplace density"); ax.legend()
plt.tight_layout(); plt.show()

mcmc_diagnostics(samples4[:, None], ["x (HMC Laplace)"],
                  title="HMC Laplace diagnostics")
ess4 = float(ess_geyer(samples4))
print(f"ESS = {ess4:.0f} ({100*ess4/len(samples4):.1f}% of N={len(samples4)})")
```

HMC acceptance rate = 1.000 (should be 1.000 up to FP rounding)



ESS = 1585 (39.6% of N=4000)

Mixing comment. The histogram lines up neatly with $\frac{1}{2}e^{-|x|}$. With an exact integrator each proposal is accepted, and $\sigma = 2$ is long enough for the trajectory to traverse several bounces, producing low autocorrelation and ESS that is a very large fraction of the post-burn-in sample size. The chain has mixed excellently.

Problem 5 — EM / MM

Part (a): MM monotonicity

By the two MM conditions,

$$F(\theta^{(t+1)}) \stackrel{(\text{minorize})}{\geq} G(\theta^{(t+1)}, \theta^{(t)}) \stackrel{(\text{argmax})}{\geq} G(\theta^{(t)}, \theta^{(t)}) \stackrel{(\text{agree})}{=} F(\theta^{(t)}).$$

Hence $F(\theta^{(t+1)}) \geq F(\theta^{(t)})$ for every t . $\square$

Part (b)(i): G minorizes $F(\theta) = \cos \theta$

Let $h(\theta) := F(\theta) - G(\theta, \theta^{(t)}) = \cos \theta - \cos \theta^{(t)} + \sin \theta^{(t)}(\theta - \theta^{(t)}) + \frac{1}{2}(\theta - \theta^{(t)})^2$.

Clearly $h(\theta^{(t)}) = 0$. Differentiating,

$$h'(\theta) = -\sin \theta + \sin \theta^{(t)} + (\theta - \theta^{(t)}), \quad h'(\theta^{(t)}) = 0,$$

$$h''(\theta) = -\cos \theta + 1 \geq 0.$$

So h is convex with global minimum 0 at $\theta^{(t)}$, hence $h(\theta) \geq 0$, i.e. $G(\theta, \theta^{(t)}) \leq F(\theta)$, with equality at $\theta^{(t)}$. $\square$

Part (b)(ii): The MM iterate

$\partial_\theta G(\theta, \theta^{(t)}) = -\sin \theta^{(t)} - (\theta - \theta^{(t)}) = 0$ gives

$$\theta^{(t+1)} = \theta^{(t)} - \sin \theta^{(t)}.$$

(Second derivative w.r.t. θ is $-1 < 0$, so this is a maximum.) $\square$

Part (b)(iii): Convergence on $(-\pi, \pi)$

Let $\phi(\theta) = \theta - \sin \theta$. For $\theta \in (0, \pi)$, $\sin \theta > 0$, so $\phi(\theta) < \theta$; also $\phi(\theta) > 0$ since $\theta > \sin \theta$ for $\theta > 0$. Hence ϕ maps $(0, \pi)$ into itself and is strictly decreasing in the sense $0 < \phi(\theta) < \theta$. The iterates $\theta^{(t)}$ starting in $(0, \pi)$ form a strictly decreasing sequence bounded below by 0; it converges to some $L \geq 0$. Taking limits in $L = L - \sin L$ gives $\sin L = 0$, and $L \in [0, \pi)$ forces $L = 0$.

By the odd symmetry $\phi(-\theta) = -\phi(\theta)$, the same argument shows convergence to 0 from any $\theta^{(0)} \in (-\pi, 0)$. At $\theta^{(0)} = 0$ the iteration is stationary at the true maximizer. $\square$

Part (c): KL nonnegativity

Using $u \log u \geq u - 1$ (equality iff $u = 1$) with $u = p(x)/q(x)$, multiply by $q(x) \geq 0$ and integrate:

$$\int p(x) \log \frac{p(x)}{q(x)} dx \geq \int (p(x) - q(x)) dx = 1 - 1 = 0.$$

Equality forces $p(x)/q(x) = 1$ a.e., i.e. $p = q$ (a.e.). $\square$

Part (d): G minorizes F for likelihood

With $F(\theta) = \log f_{Y|\theta}(y)$ and $G(\theta, \theta^{(t)}) = F(\theta) - D(f_{X|Y=y, \theta^{(t)}} \| f_{X|Y=y, \theta})$:

- $G(\theta, \theta^{(t)}) \leq F(\theta)$ because the KL term is ≥ 0 .
- $G(\theta^{(t)}, \theta^{(t)}) = F(\theta^{(t)}) - 0 = F(\theta^{(t)})$ because $D(p \| p) = 0$.

Both MM conditions hold. $\square$

Part (e): Alternative expression for G

By Bayes' rule, $\log f_{Y|\theta}(y) = \log f_{Y, X|\theta}(y, x) - \log f_{X|Y=y, \theta}(x)$. Take $E_{X|Y, \theta^{(t)}}[\cdot]$ on both sides; the left side is θ -dependent only through $F(\theta)$:

$$F(\theta) = \int f_{X|Y, \theta^{(t)}}(x) \log f_{Y, X|\theta}(y, x) dx - \int f_{X|Y, \theta^{(t)}}(x) \log f_{X|Y, \theta}(x) dx.$$

Subtract the KL divergence:

$$\begin{aligned} G(\theta, \theta^{(t)}) &= F(\theta) - \int f_{X|Y, \theta^{(t)}} \log \frac{f_{X|Y, \theta^{(t)}}}{f_{X|Y, \theta}} dx \\ &= \int f_{X|Y, \theta^{(t)}}(x) \log f_{Y, X|\theta}(y, x) dx - \int f_{X|Y, \theta^{(t)}}(x) \log f_{X|Y, \theta^{(t)}}(x) dx. \end{aligned}$$

This is exactly the claimed expression. $\square$

Part (f): MM applied here = EM

The second term in part (e) does not depend on θ , so

$$\operatorname{argmax}_{\theta} G(\theta, \theta^{(t)}) = \operatorname{argmax}_{\theta} \int f_{X|Y, \theta^{(t)}}(x) \log f_{Y, X|\theta}(y, x) dx = \operatorname{argmax}_{\theta} E(\theta, \theta^{(t)}),$$

which is the EM M-step. The E-step in EM computes the same integrand. So EM is exactly MM applied to $F(\theta) = \log f_{Y|\theta}(y)$ with the G constructed via KL. $\square$

Part (g): EM for the Bernoulli-Gaussian mixture

Model. $X_i \sim \text{Ber}(w)$ i.i.d., $Y_i | X_i = 1 \sim N(\theta_1, 1)$, $Y_i | X_i = 0 \sim N(\theta_0, 1)$. The complete-data log-likelihood is

$$\log f_{Y, X|\theta}(y, x) = \sum_i [x_i \log w + (1 - x_i) \log(1 - w) + x_i \log \phi(y_i; \theta_1) + (1 - x_i) \log \phi(y_i; \theta_0)],$$

where $\phi(\cdot; \mu)$ is the $N(\mu, 1)$ density.

E-step (responsibilities). Since X_i is discrete and independent across i given Y ,

$$\gamma_i^{(t)} := P(X_i = 1 | Y_i = y_i, \theta^{(t)}) = \frac{w^{(t)} \phi(y_i; \theta_1^{(t)})}{w^{(t)} \phi(y_i; \theta_1^{(t)}) + (1 - w^{(t)}) \phi(y_i; \theta_0^{(t)})}.$$

So

$$E(\theta, \theta^{(t)}) = \sum_i \left[\gamma_i^{(t)} \log w + (1 - \gamma_i^{(t)}) \log(1 - w) + \gamma_i^{(t)} \log \phi(y_i; \theta_1) + (1 - \gamma_i^{(t)}) \log \phi(y_i; \theta_0) \right].$$

M-step. Setting derivatives to zero:

$$w^{(t+1)} = \frac{1}{n} \sum_i \gamma_i^{(t)}, \quad \theta_1^{(t+1)} = \frac{\sum_i \gamma_i^{(t)} y_i}{\sum_i \gamma_i^{(t)}}, \quad \theta_0^{(t+1)} = \frac{\sum_i (1 - \gamma_i^{(t)}) y_i}{\sum_i (1 - \gamma_i^{(t)})}.$$

Simulation demo. True $(w, \theta_0, \theta_1) = (0.4, -1, 2)$, $n = 200$:

```
np.random.seed(238)

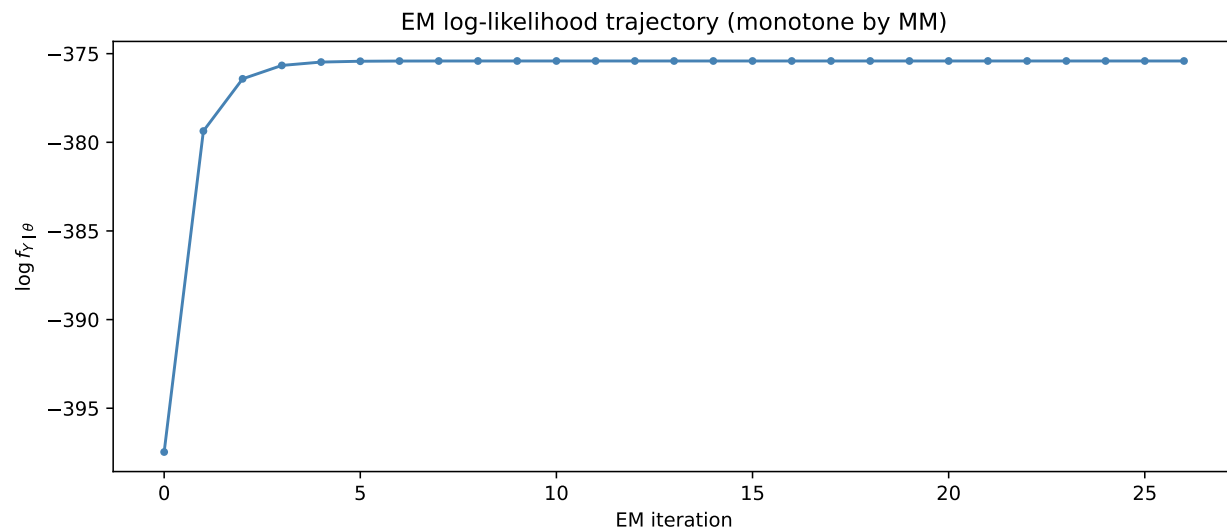
n_sim = 200
w_true, t0_true, t1_true = 0.4, -1.0, 2.0
x_sim = np.random.rand(n_sim) < w_true
y_sim = np.where(x_sim, np.random.randn(n_sim) + t1_true,
                 np.random.randn(n_sim) + t0_true)

(w_hat, t0_hat, t1_hat), hist = em_two_component(
    y_sim, w0=0.5, theta0_0=-0.5, theta1_0=0.5,
    max_iter=500, tol=1e-8)
print(f"EM converged in {len(hist)} iterations.")
print(f"  w_hat      = {w_hat:.4f}      (true {w_true})")
print(f"  theta_0_hat = {t0_hat:.4f}      (true {t0_true})")
print(f"  theta_1_hat = {t1_hat:.4f}      (true {t1_true})")

hist_arr = np.array(hist)
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(hist_arr[:, 3], marker='o', ms=3, lw=1.5, color="steelblue")
ax.set_xlabel("EM iteration"); ax.set_ylabel(r"$\log f_{Y\mid\theta}$")
ax.set_title("EM log-likelihood trajectory (monotone by MM)")
plt.tight_layout(); plt.show()
```

EM converged in 27 iterations.

```
w_hat      = 0.4178      (true 0.4)
theta_0_hat = -0.9960    (true -1.0)
theta_1_hat = 2.1584     (true 2.0)
```



The log-likelihood increases monotonically at every EM step (as the MM theorem of part (a) guarantees), and the estimates are close to the truth given only 200 samples. Exchange of the two components would give an equivalent fit; the initial values here break the symmetry towards $(\theta_0, \theta_1) = (-1, 2)$.